

IA en la Industria 4.0

Fabio Hernán Realpe Martínez

UNIVERSIDAD DEL CAUCA
INGENIERIA EN AUTOMATICA INDUSTRIAL
POPAYAN - 2026

Índice general

1. Herramientas de IA en la Industria 4.0	6
1.1. Unidad temática: Arquitecturas de Orquestación Agéntica para la Industria 4.0	6
1.1.1. La Capa de Orquestación: El Sistema Nervioso Central	6
1.1.2. MCP y el Vínculo con el Hardware	7
1.1.3. Interfaz de Usuario como Reflejo del Razonamiento	8
1.2. Que es un agente?	10
1.2.1. Memoria de Corto Plazo (Contexto Operativo):	11
1.2.2. Memoria de Largo Plazo (Historial y Patrones):	11
1.2.3. Razonamiento mediante Reglas de Negocio:	11
1.2.4. Diferenciación Técnica: El Agente como Operador Dinámico	12
1.3. LangChain	12
1.4. LangGraph	12
1.4.1. LangGraph como el "Cerebro" de la Industria 4.0	13
1.4.2. Ingeniería de Prompts: Del Texto a la Estructura de Datos	14
1.5. Tokens	15
1.6. Modelos de Chat vs. Herramientas (Tool/JSON)	18
1.7. Zod?	19
1.8. Estructura de salida	21
2. Esp32	24
2.1. Qué es PlatformIO? PlatformIO	24
2.2. Esquema del Proyecto	29
2.3. Programación de protocolos de comunicación	32
2.3.1. Configuración de variables globales para las conexión wifi	32
2.3.2. Configuración de Lectura, Escritura y Reset de parámetros.	34
2.3.3. Ciclo de Vida del MQTT	38

Índice de figuras

1.1. Arquitectura de Tres Capas para el Control Autónomo de Procesos	9
2.1. Proyecto MCP	25
2.2. Estructura del proyecto	25
2.3. Diagrama principal del proyecto	30
2.4. Ciclo de Vida de un Mensaje MQTT.	32
2.5. Arquitectura de configuración Wiffi	35
2.6. Ciclo de vida de Mqtt	39

Algoritmos

1.1. Esquema de validación Zod.	22
1.2. Respuesta del modelo.	22
2.1. platformio.ini.	26
2.2. Compilar y cargar	26
2.3. Configuración de platformio.in	27
2.4. Versionamiento	29
2.5. Configuraciones globales del dispositivo	33
2.6. mcp_settings.hpp	36
2.7. settingsRead	36
2.8. settingsReset	37
2.9. settingsSave	38
2.10. Función TaskMqttReconnect.	41
2.11. Función mqttloop().	42
2.12. Función mqtt_connect().	43
2.13. Función callback()	44
2.14. Función mqtt_response()	45
2.15. Función mqtt_publish()	46
2.16. Json()	47

Capítulo 1

Herramientas de IA en la Industria 4.0

1.1. Unidad temática: Arquitecturas de Orquestación Agéntica para la Industria 4.0

Esta propuesta académica redefine la integración de la Inteligencia Artificial en entornos productivos, estableciendo una distinción crítica entre el consumo pasivo de modelos y el diseño de sistemas autónomos. A continuación, se explican los conceptos fundamentales de esta arquitectura bajo una visión de ingeniería agéntica:

La Superación del Modelo "Wrapper" El enfoque tradicional, conocido como "ChatGPT wrapper", se limita a actuar como una interfaz superficial que reenvía consultas a un modelo lingüístico sin poseer una lógica interna o conexión real con el entorno. En contraste, un sistema agéntico industrial se concibe como un operador proactivo. Mientras que el primero es un espejo estático de las capacidades del modelo, el sistema agéntico utiliza la IA para razonar, integrando el flujo de trabajo con la realidad física de la planta.

1.1.1. La Capa de Orquestación: El Sistema Nervioso Central

El núcleo de esta arquitectura es la Orquestación, implementada mediante LangGraph. Esta herramienta funciona como el cerebro del sistema, gestionando el estado y permitiendo flujos de decisión cíclicos. A diferencia de un chat convencional, esta capa dota a la IA de memoria y razonamiento lógico sobre procesos industriales, permitiendo que el sistema no solo responda, sino que mantenga la continuidad y coherencia en tareas de larga duración o procesos repetitivos de control.

1.1.2. MCP y el Vínculo con el Hardware

Para que la inteligencia lógica (LLM) pueda interactuar con la acción física (Hardware), se utiliza el Model Context Protocol (MCP). Este protocolo actúa como la capa de herramientas, permitiendo que los agentes de IA se conecten de manera segura y estandarizada con sensores IoT como sensores y bases de datos industriales. El MCP permite que la IA "sienta" el hardware, convirtiendo señales físicas en datos procesables para el razonamiento del agente.

MCP (Model Context Protocol) es un estándar abierto desarrollado recientemente por Anthropic que está cambiando la forma en que los asistentes de IA interactúan con los datos y las herramientas ver1.1.

- 1. Introducción En el panorama actual del desarrollo con Inteligencia Artificial, la integración de modelos de lenguaje (LLMs) con fuentes de datos externas (bases de datos, archivos locales, APIs) es fragmentada. Cada integración requiere una implementación personalizada, lo que genera silos de información y deuda técnica.

Model Context Protocol (MCP) surge como un estándar abierto diseñado para reemplazar estas integraciones punto a punto con un protocolo universal y seguro.

- 2. El Problema: El "Impuesto de Integración" Hasta ahora, conectar un Agente de IA a un entorno de trabajo implicaba:

Mantenimiento constante: APIs que cambian y requieren actualizaciones manuales.

Riesgos de seguridad: Métodos de acceso a datos no estandarizados.

Falta de portabilidad: Una herramienta diseñada para un modelo no funcionaba fácilmente en otro.

- 3. La Solución: Arquitectura MCP MCP funciona bajo un modelo de arquitectura Cliente-Servidor simplificado:

Componente	Función
MCP Host	La aplicación de IA (ej. Claude Desktop, IDEs, aplicaciones personalizadas en Node.js).
MCP Client	Mantiene la conexión bidireccional con los servidores dentro del Host.
MCP Server	Microservicios ligeros que exponen datos o herramientas específicas (Filesystem, GitHub, SQL, sensores) mediante el estándar MCP.

Tabla 1.1: Cliente-Servidor MCP.

- 4. Beneficios Clave Interoperabilidad:
 - Una vez que escribes un servidor MCP (por ejemplo, para leer datos de sensores o estados de un PLC), cualquier aplicación compatible con MCP puede usarlo.

- **Control de Contexto:** El modelo solo accede a la información necesaria en el momento preciso, optimizando el uso de la ventana de contexto.
- **Seguridad Local:** Los servidores pueden ejecutarse localmente, permitiendo que la IA interactúe con datos sensibles sin que estos salgan de la infraestructura privada del usuario o empresa.

5. Casos de Uso en Desarrollo

- **Ingeniería de Software:** Acceso directo a repositorios para refactorización de código en tiempo real.
- **Sistemas Industriales / IoT:** Un servidor MCP que actúe como puente entre protocolos industriales (Modbus/OPC-UA) y el modelo de IA para diagnóstico de fallas.
- **Gestión de Datos:** Consultas a bases de datos en lenguaje natural mediante la exposición de esquemas SQL seguros a través de MCP.

6. Conclusión y Próximos Pasos

- MCP no es solo una herramienta, es el puerto estándar que permitirá que los agentes de IA dejen de ser simples generadores de texto para convertirse en asistentes operativos reales con acceso a herramientas del mundo físico y digital.

1.1.3. Interfaz de Usuario como Reflejo del Razonamiento

Finalmente, la capa de UI, desarrollada en React o React Native, deja de ser una simple ventana de chat para convertirse en un tablero de control del proceso de pensamiento del agente. En esta arquitectura, la interfaz es un reflejo fiel de un proceso de razonamiento autónomo, donde el usuario puede observar cómo el agente decide y actúa sobre los activos físicos en tiempo real.

La figura muestra el esquema funcional del sistema ver 1.1.

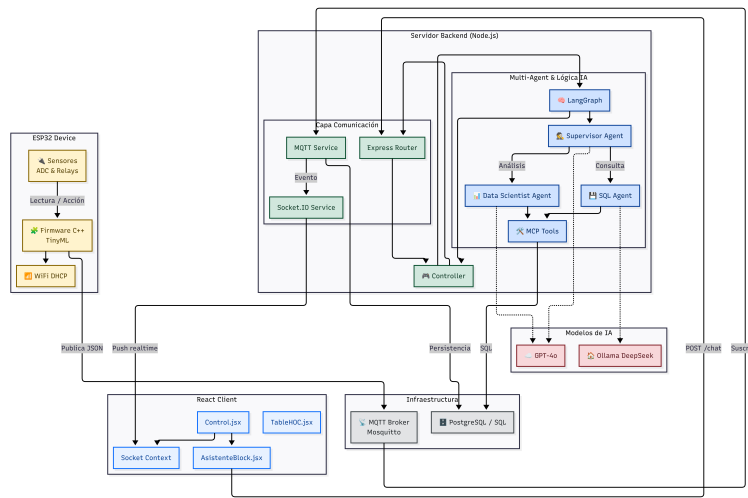


Figura 1.1: Arquitectura de Tres Capas para el Control Autónomo de Procesos

1. **Capa de Interfaz (UI): El Espejo Reactivo** En esta capa superior, el usuario interactúa con el sistema a través de una aplicación desarrollada en React Native. A diferencia de una aplicación convencional, esta interfaz no es solo un chat, sino un panel de control que refleja el razonamiento de la IA en tiempo real.
 - **WebSockets / Socket.io:** Actúan como el puente de baja latencia que permite que el agente de IA envíe alertas o actualizaciones de sensores de forma instantánea a la pantalla del operador sin necesidad de recargar.
2. **Capa de Orquestación: El Cerebro (LangGraph)** Esta es la capa que trasciende el concepto de "wrapper". Aquí, LangGraph funciona como el sistema nervioso central que define la lógica de control.
 - **Estado y Memoria:** El sistema mantiene un registro de lo que ha sucedido en la planta, permitiendo que la IA tome decisiones basadas en el historial y no solo en el último mensaje recibido.
 - **Flujos Cíclicos:** Permite que el agente realice bucles de razonamiento: "Leo el sensor - > Analizo - > Si el valor es incorrecto, intento ajustar - > Vuelvo a leer".
3. **Capa de Models & Tools: El Vínculo con la Realidad** Aquí es donde la inteligencia abstracta se encuentra con la ejecución física mediante el Model Context Protocol (MCP).

Servidor MCP: Es el estándar que permite al agente de IA "ver" y "tocar" el hardware. Expone las capacidades de la planta como herramientas que el modelo puede usar.

Protocolos Industriales:

- MQTT: Utilizado para la telemetría masiva de sensores IoT (como el sensor NPK de suelo), permitiendo que el agente reciba datos de forma eficiente.
- OPC-UA: Garantiza la interoperabilidad con maquinaria industrial pesada y servidores de planta estándar.
- Modbus: Permite el control directo sobre PLCs y dispositivos que manejan la acción física (actuadores, motores).

4. Inteligencia y Datos LLM

- (Llama3 / Ollama): Es el motor de inferencia que procesa el lenguaje natural y los datos de los sensores para generar planes de acción.
- Bases de Datos (InfluxDB / Postgres): Almacenan el histórico de telemetría y los metadatos de la operación para que el agente pueda realizar análisis de tendencias o mantenimiento predictivo.

Conclusión del Flujo: Cuando un sensor reporta una anomalía vía MQTT, el Servidor MCP lo comunica al Grafo de Orquestación. El LLM analiza la situación, decide una acción correctiva (ej. ajustar una válvula vía Modbus) y actualiza simultáneamente la UI del operador mediante Sockets, transformando la planta mediante un ciclo cerrado de percepción y acción.

Para que el sistema sea eficiente, es vital distinguir cómo se procesa la información dentro de la orquestación:

1. Chain (Cadena): Sigue pasos fijos y predeterminados sin tomar decisiones propias. Se recomienda usarlas primero para tareas simples y lineales.
2. Agent (Agente): Tiene la capacidad de elegir el siguiente paso basándose en el contexto. Decide "qué hacer a continuación" para resolver problemas complejos, siempre bajo el control del desarrollador.

1.2. Que es un agente?

En esencia, un Agente de IA se define como la combinación de un LLM + Memoria + Herramientas + Reglas. Mientras que un wrapper es un espejo estático que solo refleja lo que la IA dice, un sistema agéntico industrial es un operador dinámico que utiliza estos componentes para transformar la planta:

Memoria e Integración: Gracias a la orquestación, el agente puede "pensar antes de actuar". Por ejemplo, si una regla de negocio dicta que no se debe aumentar la velocidad de una banda transportadora si la temperatura del motor supera un umbral crítico, el agente verifica el sensor mediante sus herramientas, razona la restricción usando su memoria y decide cancelar la acción, informando al operador el porqué de su decisión a través de la UI.

1.2.1. Memoria de Corto Plazo (Contexto Operativo):

El agente retiene los datos inmediatos del flujo de trabajo, como el estado actual de una línea de producción o la última lectura de un protocolo OPC-UA. Esto le permite "pensar" si la acción solicitada es coherente con el estado presente de la máquina.

1.2.2. Memoria de Largo Plazo (Historial y Patrones):

Mediante la conexión con bases de datos como InfluxDB o PostgreSQL, el agente accede al histórico de fallos o métricas de rendimiento pasadas. Por ejemplo, si el agente detecta una vibración inusual, consulta su memoria para verificar si ese patrón precedió a una avería el mes anterior, activando un diagnóstico preventivo en lugar de una simple alerta.

En un sistema robusto, la memoria de largo plazo no vive dentro del modelo (LLM), sino que es una herramienta externa que el agente consulta a través de la capa de orquestación.

Su funcionamiento se divide en tres ejes:

1. Persistencia de Experiencias (Historial de Operación): A través de bases de datos como PostgreSQL o InfluxDB, el agente guarda cada evento crítico que ocurre en la planta. Si un motor presenta una vibración inusual, el agente consulta su memoria de largo plazo para verificar: "¿Este patrón de vibración ha ocurrido antes? ¿Terminó en una falla crítica la última vez?". Esto permite que el agente razone basándose en la experiencia acumulada.
2. RAG (Generación Aumentada por Recuperación) - El "Manual Técnico": La memoria de largo plazo también contiene el conocimiento estático: manuales de maquinaria, protocolos de seguridad y normativas industriales. Mediante técnicas de RAG, el agente busca en milisegundos la solución técnica a un problema detectado por los sensores, integrando ese conocimiento en su proceso de decisión.
3. Memoria Semántica vs. Memoria de Episodios:
 - Episódica: Recuerda interacciones pasadas con el operador o estados específicos de la planta.
 - Semántica: Comprende los conceptos técnicos y las relaciones entre los componentes de la industria 4.0.

1.2.3. Razonamiento mediante Reglas de Negocio:

La orquestación (vía LangGraph) permite al agente aplicar lógica de control compleja antes de actuar. Si una regla dicta que no se debe aumentar la velocidad de una banda transportadora si la temperatura del motor supera un umbral crítico, el agente verifica el sensor, razona la restricción y decide cancelar la operación, informando al operador el motivo técnico a través de la UI en React Native.

1.2.4. Diferenciación Técnica: El Agente como Operador Dinámico

Mientras que un wrapper es un espejo estático que solo refleja lo que la IA dice sin comprender el entorno, un sistema agéntico industrial se define como la combinación de LLM + Memoria + Herramientas + Reglas. Este sistema utiliza:

1. La IA para Razonar: Procesa datos complejos y lenguaje natural para entender objetivos de alto nivel.
2. La Orquestación para Decidir: Determina el "siguiente paso" lógico basándose en el estado y la memoria.
3. Las Herramientas (MCP) para Transformar: Actúa directamente sobre el entorno físico mediante protocolos como MQTT o Modbus, cerrando el ciclo entre el pensamiento digital y la acción mecánica.

En conclusión, la integración total de la memoria permite que el agente pase de ser un consultor pasivo a un operador dinámico capaz de gestionar la complejidad de la Industria 4.0, asegurando que cada acción física sea el resultado de un análisis histórico y situacional profundo.

1.3. LangChain

Dentro de la arquitectura de tres capas, LangChain se posiciona como el conjunto de herramientas (toolkit) fundamental para construir flujos de trabajo avanzados con modelos de lenguaje, operando específicamente como el motor interno de la capa de orquestación. Su función principal es servir como el "cableado" lógico que permite integrar y gestionar componentes críticos como prompts, modelos, herramientas y técnicas de RAG (Generación Aumentada por Recuperación).

Al utilizar el lenguaje de expresión de LangChain (LCEL), el sistema trasciende las aplicaciones lineales para convertirse en una estructura capaz de decidir "qué hacer a continuación" basándose en el contexto y las reglas de negocio establecidas. En el ámbito de la Industria 4.0, LangChain permite "cablear" la inteligencia lógica con la acción física, facilitando que el agente utilice su memoria para analizar datos históricos y razonar antes de ejecutar cualquier operación sobre el hardware. Así, LangChain transforma a la IA en un operador dinámico que no solo genera texto, sino que orquesta acciones precisas que transforman el entorno productivo de la planta.

1.4. LangGraph

LangGraph es una librería construida sobre los bloques de LangChain que permite definir flujos de trabajo mediante grafos. Se sitúa jerárquicamente por

encima de LangChain, utilizando sus componentes (como modelos y herramientas) para orquestar comportamientos más sofisticados.

Conceptos Clave de Operación

Para implementar una orquestación robusta, debe dominar los siguientes pilares de LangGraph:

1. **Nodos y Aristas (Nodes & Edges):** El flujo se define como un grafo donde los nodos representan funciones de procesamiento y las aristas dictan el camino hacia el siguiente paso.
2. **Estado (State):** LangGraph mantiene un objeto de estado compartido que persiste a través de los nodos. En la industria, esto permite que el agente "recuerde" lecturas de sensores previas durante un ciclo de decisión.
3. **Ciclos y Reintentos (Retries):** A diferencia de las cadenas simples, permite bucles de retroalimentación. Si un sensor reporta un error de lectura, el grafo puede reintentar la conexión o ejecutar una rutina de limpieza automáticamente.
4. **Human-in-the-loop (HITL):** Es la capacidad de pausar la ejecución del agente para esperar la aprobación o corrección de un operador humano antes de ejecutar una acción física crítica en la planta.

1.4.1. LangGraph como el "Cerebro" de la Industria 4.0

En el contexto, LangGraph es lo que permite que el sistema deje de ser un "ChatGPT wrapper" (un espejo estático de texto) y se convierta en un operador dinámico.

1. En el nivel superior de la lógica se encuentra LangGraph, que actúa como el "sistema nervioso central" de la aplicación. Su función es definir el grafo de control, el estado y el flujo de trabajo del agente.

Gestión de Estados: A diferencia de un chat convencional, LangGraph mantiene la memoria y el estado del proceso, permitiendo ciclos de reintento y persistencia de datos industriales.

Toma de Decisiones: Es aquí donde el sistema decide "qué hacer a continuación" basándose en el contexto operativo y las reglas de negocio predefinidas.

Intervención Humana: Permite integrar bucles de "Human-in-the-loop" (HITL), pausando acciones críticas en la planta para esperar la aprobación de un operador.

2. **Capa de Ejecución (LangChain)** LangGraph se apoya en los bloques de construcción de LangChain para ejecutar tareas específicas dentro de la orquestación. LangChain funciona como un "toolkit" que gestiona:

Componentes: Manejo de prompts, modelos de lenguaje (LLMs) y técnicas de RAG (Generación Aumentada por Recuperación) para consultar manuales técnicos o históricos.

LCEL: Utiliza el lenguaje de expresión de LangChain para "cablear" de forma eficiente la lógica con las herramientas de hardware.

3. Capa de Percepción y Acción (Models & Tools) En la base de la pirámide se encuentran los Modelos, Bases de Datos y herramientas de Hardware. Esta capa permite que la IA deje de ser un "wrapper" estático y se convierta en un operador dinámico:

Percepción: A través de protocolos como MCP, MQTT u OPC-UA, el agente recibe telemetría en tiempo real desde los sensores de la planta.

Acción: El agente utiliza herramientas para transformar el entorno físico, ejecutando comandos sobre PLCs o actuadores.

Memoria e Inteligencia: Combina el razonamiento del LLM con la memoria de largo plazo almacenada en bases de datos para realizar mantenimiento predictivo.

4. En conclusión, el sistema funciona como un ciclo cerrado donde la UI (paneles o chats) es solo el reflejo de un proceso de razonamiento autónomo orquestado por LangGraph, ejecutado por LangChain y materializado a través de herramientas industriales conectadas.

1.4.2. Ingeniería de Prompts: Del Texto a la Estructura de Datos

Tratar la salida del LLM como datos, no como párrafos Cuando usas ChatGPT como usuario, esperas una conversación. Pero cuando eres desarrollador, los párrafos son un problema porque contienen "ruido" (palabras de relleno, cortesía, explicaciones innecesarias).

1. "Trata la salida del LLM como datos, no como párrafos"
 - Normalmente, usamos modelos de lenguaje (LLMs) para escribir correos o ensayos. Sin embargo, en el desarrollo de software moderno, queremos que la IA sea un componente de nuestra aplicación.
 - El problema: El texto libre (párrafos) es difícil de procesar para un programa.
 - La solución: Pedirle a la IA que responda estrictamente en formato JSON. Esto permite que el sistema "entienda" la respuesta automáticamente sin necesidad de que un humano la lea.
2. "JSON = predecible, testeable e integrable" ¿Por qué JSON? Porque es el lenguaje universal de la web.

- Predecible:: Sabes exactamente dónde estará cada dato (ej. el precio estará siempre en la clave "precio").
- Testeable: Puedes escribir pruebas automáticas para verificar si la IA está respondiendo correctamente.
- Integrable: Los datos pueden pasar directamente a una base de datos o a una interfaz de usuario sin pasos intermedios complicados.

3. Aplicar estandar Zod

- Este es el punto más técnico y crítico para la estabilidad del sistema. Zod es una librería de validación de esquemas que actúa como un contrato.
- El Riesgo: Las IAs a veces "alucinan" o fallan en el formato (por ejemplo, ponen un texto donde debería ir un número).
- La Solución (Zod): Defines un contrato estricto. Si la IA devuelve algo que no encaja exactamente con la "forma" definida (por ejemplo, falta un campo obligatorio), Zod lanza un error inmediatamente.
- Resultado: Tu aplicación nunca procesará datos corruptos o malformados, lo que evita errores inesperados en producción.

1.5. Tokens

Dado que no siempre se puede contar tokens exactos a ojo, se utiliza una aproximación estadística para el lenguaje humano ver 1.1.

$$Tokens \approx \frac{Palabras}{0,75} \quad (1.1)$$

1. Aplicación: Si tienes un documento de 1,500 palabras, puedes estimar que consumirá unos 2,000 tokens.
2. Nota técnica: Para código de programación (como el que usas en Node.js o React), la relación cambia porque los espacios y símbolos cuentan más; ahí la relación suele ser casi 1 : 1.

Para presupuestar una aplicación, se calcula el costo por cada mil tokens (MTok) ver 1.2.

$$Costo_{total} = (Tokens_{entrada} \times Tarifa_{input}) + (Tokens_{salida} \times Tarifa_{output}) \quad (1.2)$$

Dato clave: Casi todos los modelos (como GPT-4 o Claude) cobran más caro por los tokens que la IA genera (output) que por los que tú le envías (input).

3. Límite de la Ventana de Contexto

La ventana de contexto es un recurso finito. El cálculo que debes hacer para que tu aplicación no "falle" es ver 1.3:

$$Contexto_{libre} = Ventana_{máxima} - (Tokens_{instrucciones} + Tokens_{datos} + Tokens_{histórial}) \quad (1.3)$$

Truncamiento: Si el resultado es negativo, el modelo truncará la información y perderá coherencia.

Optimización: Aquí es donde entran tus proyectos de agentes de IA; un buen agente debe decidir qué información "olvidar" o resumir para no saturar este cálculo.

El término Temperature (Temperatura)

Es un hiperparámetro crucial que controla el grado de aleatoriedad o "creatividad" de las respuestas de un modelo de lenguaje. En términos técnicos, la temperatura ajusta las probabilidades de las palabras (tokens) que el modelo considera para su siguiente respuesta. No cambia el conocimiento de la IA, sino qué tan "arriesgada" es al elegir la siguiente palabra.

- Valores bajos (0.1 - 0.3): El modelo se vuelve determinista. Elige siempre la palabra más probable. Es ideal para tareas que requieren precisión, como escribir código, resolver problemas matemáticos o extraer datos en formato JSON.
- Valores altos (0.7 - 1.2+): El modelo se vuelve creativo y diverso. Empieza a elegir palabras menos obvias, lo que genera respuestas más variadas, poéticas o sorprendentes. Es ideal para lluvia de ideas o escritura creativa.

El Cálculo Matemático Detrás

Para entender cómo soporta esto la información de tus diapositivas anteriores, la temperatura actúa sobre una función llamada Softmax. Si el modelo tiene varias opciones para la siguiente palabra con sus respectivas "puntuaciones" (z_i), la fórmula de probabilidad P_i ajustada por la temperatura (T) es ver 1.4:

$$P_i = \frac{\exp(z_i/T)}{\sum(z_j/T)} \quad (1.4)$$

- Si T es alta: Las diferencias entre las probabilidades se "aplanan", haciendo que palabras menos probables tengan una oportunidad real de ser elegidas.
- Si T es baja: Las diferencias se exageran, haciendo que la opción más probable domine por completo.

Dado que se está trabajando en agentes industriales y sistemas agénticos que deben usar herramientas y razonar, la configuración de la temperatura es vital:

1. Para el Agente de Mantenimiento o PLCs: Deberías usar una temperatura cercana a 0. No quieres que la IA "sea creativa" al decidir si una máquina tiene un fallo en Factory I/O; necesitas una respuesta lógica y predecible basada en datos.
2. Para el Agente con o un agente de respuesta conversacional: Podrías usar una temperatura media (0,7) para que la conversación se sienta natural, fluida y no siempre responda lo mismo, dándole "personalidad" al agente.

La eficiencia de un sistema de IA no solo depende de los datos, sino de la gestión inteligente del contexto. Cada interacción suma información proveniente de diversas fuentes: las instrucciones del sistema, la entrada del usuario, el historial de la conversación y las respuestas de las herramientas externas. Mantener este flujo corto y conciso es fundamental, ya que el aumento de tokens no solo incrementa el costo financiero, sino que eleva la latencia, haciendo que el agente sea más lento al procesar y generar respuestas.

Anatomía del Contexto (Lo que el modelo "lee")

Para los proyectos de agentes industriales y aplicaciones móviles, debes considerar que la "ventana de contexto" se llena con:

- System Instructions: Las reglas base que definen el comportamiento del agente.
- User Input: Lo que el usuario escribe o solicita en el momento.
- Chat History: Los mensajes anteriores que permiten que la IA tenga memoria de la conversación.
- Tools Responses: Los datos crudos que regresan tus herramientas (por ejemplo, el estado de un sensor o el log de un PLC).
- Its own output: La respuesta que la IA acaba de generar también ocupa espacio para la siguiente interacción.

Impacto en el Rendimiento y Costo

Hay una relación directa entre volumen y velocidad:

1. Más tokens = Más que leer: Aumenta el tiempo que el modelo tarda en "entender" el contexto antes de empezar a responder.
2. Más tokens de salida = Más que generar: La generación de texto es la parte más lenta del proceso. A mayor longitud, mayor será la latencia (el tiempo de espera del usuario).
3. Latencia y Costo: Son los dos grandes enemigos. Un contexto saturado hace que tu aplicación se sienta pesada y que el consumo de la API se dispare innecesariamente.

Regla de Oro para el Desarrollador

"Short and concise > long, random dumping" (Corto y conciso es mejor que volcar información larga y aleatoria). En lugar de enviarle a la IA todos los datos históricos de un sensor, es mejor enviarle solo un resumen o los últimos 5 minutos de lectura. Esto garantiza que el agente sea rápido, preciso y económico.

Ajuste de Precisión: Temperatura y Parámetros

Aquí es donde configuras "la personalidad" de tu agente:

- Temperature (Estabilidad vs. Creatividad):
Baja (0.1 - 0.2): Respuestas lógicas y consistentes. Ideal para tus PLCs y agentes industriales.
Alta (0.8+): Respuestas variadas. Ideal para el Tamagotchi o conversaciones fluidas.
- Top_P: Controla qué tan amplio es el abanico de palabras que la IA considera. Ayuda a filtrar opciones irrelevantes.
- Max_Tokens: Pone un límite estricto al largo de la respuesta para ahorrar dinero y evitar que la IA hable de más.
- Penalties (Penalizaciones): Evitan que la IA se vuelva repetitiva y use las mismas palabras una y otra vez

1.6. Modelos de Chat vs. Herramientas (Tool/JSON)

No todos los modelos de IA deben usarse de la misma manera. La elección depende de si necesitas una conversación humana o una ejecución técnica precisa ver 1.2.

1. Chat models: free-form answers

Los modelos de chat están optimizados para la fluidez y la naturalidad.

Uso: Respuestas en formato de texto libre (párrafos).

Ideal para: Tu proyecto de asesoramiento, donde lo importante es la personalidad y la variación en el lenguaje.

2. Modelos de Herramientas (Tools/JSON)

Estos modelos están entrenados específicamente para seguir reglas estructurales estrictas.

Uso: Generan código, llaman a funciones (functions calling) o devuelven datos en JSON.

Ideal para: Tus proyectos de PLCs, sensores y agentes de mantenimiento. Aquí no necesitas que la IA sea "simpática", sino que te entregue un dato exacto que tu aplicación pueda procesar sin errores.

3. Elige según la tarea, no por la moda: Es una advertencia sobre la ingeniería eficiente. No siempre el modelo más grande o más famoso es el mejor para tu tarea específica.

Eficiencia: Para leer un sensor o detectar un fallo, un modelo pequeño y especializado en JSON (como Gemini Flash) será mucho más rápido y económico que un modelo gigante de chat generalista.

Los modelos mostrados en la tabla son ideales para tus sistemas agénticos industriales, donde la IA debe razonar, decidir el siguiente paso y usar herramientas.

Estos modelos son ideales para tus sistemas agénticos industriales, donde la IA debe razonar, decidir el siguiente paso y usar herramientas.

Categoría	Modelo Recomendado	Ejecución	Fortaleza	Tarea Ideal
Agéntico / Potencia	Gemini 2.5 Pro	Nube	Ventana de contexto masiva para leer manuales extensos de PLCs	Orquestación de toda la planta y toma de decisiones complejas.
Costo-Eficiente	GPT-5 mini / Gemini 2.5 Flash	Nube	Alta velocidad y costo extremadamente bajo por token	Procesamiento de formularios de entrada y reportes rápidos de sensores.
Edge (Local)	Phi-4 / Llama 3.2 (3B)	Local (Raspberry Pi)	Muy ligero; corre sin internet en tus dispositivos maestros/esclavos	Control en tiempo real de sensores y lógica de PLC Industrial Shields
Visión	Qwen2.5-VL	Nube/Híbrido	Capacidad avanzada para entender mapas y coordenadas satelitales.	Conteo de plántulas y localización de árboles mediante imágenes.
Razonamiento Técnico	DeepSeek V3	Nube	Excelente en matemáticas y lógica de programación estructurada.	Generación de scripts de automatización y depuración de errores en Factory I/O.

Tabla 1.2: Modelos Agénticos (Para toma de decisiones y orquestación).

1.7. Zod?

Zod es una librería de declaración y validación de esquemas para TypeScript y Node.js. En el contexto de los LLMs, se utiliza para definir un "contrato" de datos. Si la IA es el motor que genera información, Zod es el inspector de calidad que asegura que esa información tenga la forma exacta que tu código espera.

1. ¿Cómo funciona en tu modelo de Agente? Cuando trabajas con agentes que controlan hardware (como tus PLCs o sensores NPK), no puedes permitir

- Los modelos de IA pueden generar información falsa o campos inexistentes por su naturaleza probabilística.
 - a) Filtro de precisión: Actúa como una aduana que bloquea cualquier dato que no cumpla con el formato exigido.
 - b) Consistencia de datos: Garantiza que, si pides el estado de un sensor, recibas un dato procesable y no una explicación textual innecesaria.
6. El Esquema como Fuente Única de Verdad Un contrato de tiempo de ejecución unifica la intención del programador con el comportamiento de la IA:
- Sin duplicidad de lógica: No hace falta programar validaciones manuales complejas; el esquema define las reglas y las aplica en el momento en que llega la información.
 - Confiabilidad: Transforma una salida de IA "creativa" en un componente de software predecible, permitiendo que cualquier aplicación consuma estos datos con total confianza en su formato y contenido.

Aplicación para tu agente de mantenimiento industrial, esta estructura significa que:

- Defines un contrato con Zod para los fallos del PLC.
- Usas un Tool Model para que la IA elija qué herramienta de reparación usar.
- Controlas los Tokens para que el historial no sature la memoria del proceso.

1.8. Estructura de salida

Es la técnica para obligar a un modelo de IA a responder con un formato de datos estricto en lugar de texto libre.

1. Pedir al modelo que complete un esquema, no que adivine el formato: En lugar de darle libertad creativa, le entregas un molde o "esquema" predefinido (como los esquemas de Zod vistos anteriormente) para que la IA solo rellene los huecos con la información correspondiente.
2. Uso de funciones nativas del proveedor: Para lograr esta precisión, se utilizan herramientas integradas de los modelos como el Modo JSON o el Tool Calling (llamada a funciones). Estas funciones internas aseguran que la respuesta cumpla con las reglas sintácticas del formato de datos elegido.
3. Mayor fiabilidad que un simple "por favor, responde en JSON": Esta es la mejora técnica más importante. Simplemente pedirle a una IA que use JSON en un mensaje de chat suele fallar (la IA puede añadir texto extra

o cometer errores de formato). Al usar Salidas Estructuradas, el sistema garantiza que el resultado sea siempre un objeto de datos válido que tu aplicación puede procesar de inmediato sin romperse.

El Esquema (El "Molde")

1. Primero, se define la estructura que necesitas usando un contrato como Zod ver 1.1:

Algoritmo 1.1 Esquema de validación Zod.

TypeScript

```
const InformeMotorSchema = z.object({
  id_motor: z.string(),
  temperatura_celsius: z.number(),
  estado: z.enum(["operativo", "mantenimiento_requerido", "critico"]),
  piezas_a_revisar: z.array(z.string()),
  confianza_diagnostico: z.number().min(0).max(1) });
```

2. La Entrada (Input) Se envía a la IA generalmente un texto desordenado o un log de sensores:

"El motor MTR-204 está operando a 85 grados. Se nota una vibración inusual en los rodamientos y la banda de distribución parece gastada. El técnico sugiere revisión inmediata."

3. La Salida Estructurada (Output) debido a la configuración 1.1 el modelo en Modo JSON o Tool Calling, la IA ignora las reglas de cortesía y devuelve estrictamente esto ver 1.2:

Algoritmo 1.2 Respuesta del modelo.

Json

```
{
  "id_motor": "MTR-204",
  "temperatura_celsius": 85,
  "estado": "mantenimiento_requerido",
  "piezas_a_revisar": ["rodamientos", "banda de distribución"],
  "confianza_diagnostico": 0.95
}
```

¿Por qué esto es mejor que el chat normal? Sin "adivinanzas": El modelo no intenta adivinar el formato; rellena el esquema que tú le diste.

- Integración inmediata: Tu aplicación en Node.js o React puede leer ese objeto JSON y encender una alerta roja automáticamente sin que un humano intervenga.
- Fiabilidad total: Si la IA intenta enviar algo que no sea un número en `temperatura_celsius`, el sistema (vía Zod) lo rechaza antes de que cause un error en tus bases de datos

Capítulo 2

Esp32

2.1. Qué es PlatformIO? PlatformIO

Es un ecosistema para el desarrollo de sistemas embebidos (como el ESP32) que funciona como una extensión dentro de Visual Studio Code. A diferencia de herramientas básicas, este permite gestionar automáticamente las librerías y las placas, lo cual es ideal para proyectos complejos como el control de procesos industriales o asistentes de IA.

1. Instalación Paso a Paso Descargar VS Code:

Instala Visual Studio Code en tu ordenador.

Instalar la Extensión: Abre VS Code, ve al icono de extensiones (cuadrados en la barra lateral) y busca "PlatformIO IDE". Haz clic en instalar.

Reiniciar: Una vez instalado, aparecerá un pequeño icono de una hormiga en la barra lateral izquierda.

2. Creación de un Nuevo Proyecto Al hacer clic en el icono de la hormiga y seleccionar "Open" > "New Project", deberás completar tres campos clave:

Nombre: Elige un nombre para tu proyecto (ej. Control_Autonomo_ESP32).

Placa (Board): Para la mayoría de los módulos comunes, selecciona Espressif ESP32 Dev Module.

Framework: Selecciona Arduino, que es el más amigable para principiantes ver2.1.

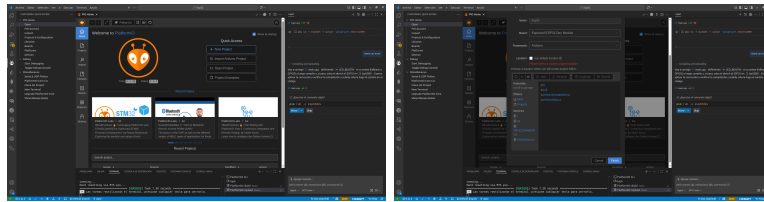


Figura 2.1: Proyecto MCP

3. Estructura del Proyecto Un proyecto de PlatformIO se organiza de forma lógica para evitar el desorden:

src/: Aquí es donde escribes tu código principal (archivo main.cpp).

lib/: Para tus propias librerías locales.

platformio.ini: Este es el archivo de configuración o "cerebro" del proyecto ver 2.2.

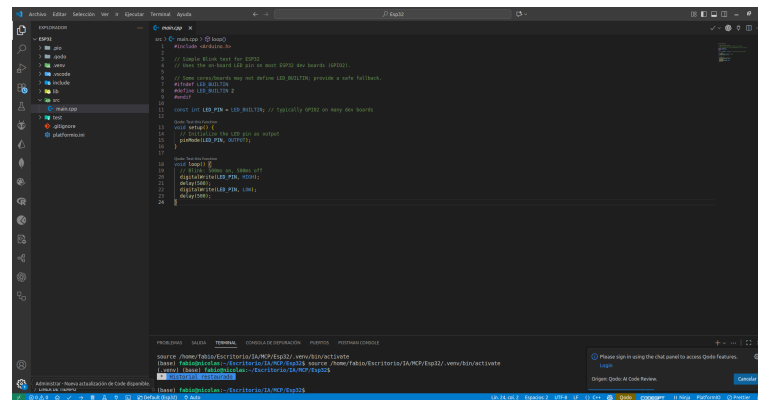


Figura 2.2: Estructura del proyecto

4. El Archivo platformio.ini, es un archivo de texto donde le dices a PlatformIO cómo debe tratar a tu placa. Un ejemplo estándar se ve así 2.1:

Algoritmo 2.1 platformio.ini.

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200 ;
upload_port = /dev/ttyUSB0
Configuración para ver datos en consola
```

5. Para compilar y cargar el programa por terminal puede escribir las siguientes líneas de código ver.

Algoritmo 2.2 Compilar y cargar

```
#1 pio run -e esp32dev
#2 pio run -e esp32dev -t upload
#3 ls -l /dev/ttyUSB* /dev/ttyACM* /dev/serial/by-id || true
#4 pio run -e esp32dev -t upload --upload-port /dev/ttyUSB0
```

- a) Compilación del Proyecto `pio run -e esp32dev`
- ¿Qué hace?: Compila (Build) el código fuente del proyecto.
 - Detalle: La opción `-e esp32dev` le indica a PlatformIO que use el entorno específico llamado `esp32dev` definido en tu archivo `platformio.ini`. Sirve para verificar si hay errores de sintaxis sin necesidad de conectar la placa.
- b) Subida del Código (Automática) `pio run -e esp32dev -t upload`
- 1) ¿Qué hace?: Compila el código y luego intenta subirlo (Flash) a la placa conectada.
 - 2) Detalle: El parámetro `-t upload` (target upload) activa el protocolo de transferencia. PlatformIO buscará automáticamente en qué puerto USB está conectada tu placa.
- c) Identificación de Puertos (Solo Linux/macOS) `ls -l /dev/ttyUSB* /dev/ttyACM* /dev/serial/by-id || true`
- 1) ¿Qué hace?: Lista todos los dispositivos seriales y USB conectados al sistema.
 - 2) Detalle: Es una herramienta de diagnóstico. Si tu placa no aparece aquí, es probable que el cable USB esté fallando o falten los drivers (CP2102/CH340). El `|| true` evita que la terminal marque un error si no encuentra ningún dispositivo.
- d) Subida Forzada a un Puerto Específico `pio run -e esp32dev -t upload --upload-port /dev/ttyUSB0`

- 1) ¿Qué hace?: Obliga a PlatformIO a subir el código a un puerto específico, en este caso /dev/ttyUSB0.
- 2) Detalle: Se usa cuando tienes varios dispositivos conectados al mismo tiempo (como dos ESP32 o un sensor NPK USB) y quieres asegurarte de que el código vaya exactamente a la placa de control de tu arquitectura.

6. La configuración del archivo para nuestro proyecto se muestra en

Algoritmo 2.3 Configuración de platformio.in

```
;MCP PlatformIO configuration file
; Archivo de configuración de PlatformIO con explicaciones en cada línea
; [platformio] (Ajustes Globales)
; Es la sección de configuración general del proyecto.
; Todo lo que pongas aquí afecta a todo el archivo platformio.ini,
; sin importar qué placa o entorno estés usando.
; Para qué sirve: Define rutas de carpetas (como data_dir)
; o cuál de todos tus entornos debe ser el prioritario por defecto (default_env).
[platformio]
default_env: entorno que se usará por defecto si no se especifica con -e
default_env = esp32dev
data_dir: carpeta donde se almacenan datos del proyecto (ej. archivos para upload)
data_dir = src/data
[common] (Sección de Herencia/Plantilla)
; Esta no es una sección oficial de PlatformIO,
; sino una convención que usamos los desarrolladores para no repetir código.
; Para qué sirve: Es como una "bolsa" donde guardas configuraciones que quieres usar en múltiples placas.
; Por ejemplo, si tienes 5 modelos de ESP32 diferentes, defines la versión o las librerías
; aquí una sola vez y luego las "llamas" desde los otros entornos usando ${common.nombre_de_variable}.
; Dato curioso: Puedes llamarla [common], [config] o como quieras, mientras luego la referencias correctamente.
[common]
version: bandera de preprocesador que se añade a la compilación (ej. para identificar builds)
version = -D BUILD_TAG=V1.0.a0-Build-20260202
lib_deps: dependencias comunes de librerías (vacío si no hay dependencias compartidas)
lib_deps =
; [env:nombre_del_entorno] (Configuración del Entorno)
; Esta es la sección más importante. Cada bloque que empiece con env: define una tarea de compilación específica.
; Para qué sirve: Aquí especificas los "fierros" (el hardware).
; Le dices a PlatformIO: "Para este entorno llamado 'esp32dev',
; usa estas librerías, esta velocidad de subida y este framework".
; Multi-entorno: Puedes tener varios. Por ejemplo: [env:debug] (para pruebas lentas con muchos mensajes).
; [env:release] (para la versión final optimizada).
[env:esp32dev]
platform: plataforma de desarrollo (núcleo y toolchain), aquí Espressif32 para ESP32
platform = espressif32
framework: framework usado por el proyecto (Arduino en este caso)
framework = arduino
board: placa objetivo definida por PlatformIO (define pines, flash, etc.)
board = esp32dev
board_build.mcu: identificador del microcontrolador (opcional, usado para algunos ajustes)
board_build.mcu = esp32
board_build.partitions:
ruta a una tabla de particiones personalizada (vacía = usar la por defecto)
board_build.partitions = default_ota.csv
upload_protocol: protocolo/utilidad que se usa para subir el firmware (esptool es común en ESP32)
upload_protocol = esptool
lib_deps: dependencias de librerías específicas de este entorno; aquí se incluye lo común
lib_deps =
    ${common.lib_deps}
; src_build_flags: flags de compilación/definiciones para el código fuente (usa la variable common.version)
src_build_flags = ${common.version}
upload_speed: baudrate usado por la herramienta de subida (esptool)
upload_speed = 921600
monitor_speed: baudrate para el monitor serie
monitor_speed = 115200
upload_port: puerto serie a usar para subir el firmware.
; Si se deja comentado, PlatformIO intentará autodetectar el puerto.
upload_port = /dev/ttyUSB0
monitor_port: puerto a usar para el monitor serie (comentado para autodetección)
monitor_port = /dev/ttyUSB0
```

7. Archivos de distribución de memoria

Nombre	Tipo	Subtipo	Offset(inicio)	Tamaño(hex)	Tamaño aprox	¿Para qué sirve?
nvs	data	nvs	0x9000	0x5000	20KB	Guarda configuraciones persistentes (WiFi, parámetros, calibración)
otadata	data	ota	0xE000	0x2000	8KB	Indica qué firmware OTA está activo
ota_0	app	ota_0	0x10000	0x1E0000	$\approx 1,9MB$	Firmware principal (aplicación)
ota_1	app	ota_1	0x1F0000	0x1E0000	$\approx 1,9MB$	Firmware de respaldo / actualización OTA
spiffs	data	spiffs	0x3D0000	0x6000	24 KB	Sistema de archivos (logs, JSON, datos)

Tabla 2.1: Particiones de Esp32

a) El ESP32 tiene memoria flash (por ejemplo 4 MB). Esta tabla le dice al sistema:

- dónde guardar configuraciones
- dónde va el programa
- dónde va el firmware OTA
- dónde guardar archivos (SPIFFS)

b) NVS = Non-Volatile Storage

- Guarda configuraciones persistentes
- Variables que no se borran al apagar
- Ej: WiFi, credenciales, calibraciones, parámetros
- Preferences
- WiFi.begin()

c) otadata

- Aquí el ESP32 guarda cuál firmware OTA está activo
- Indica si está corriendo ota0 u ota1
- Sin esto, OTA no funciona
 - ota0:
 - Primer espacio para tu programa principal
 - Aquí se graba el firmware
 - Tamaño $\approx 1,9MB$
 - Es el firmware activo o uno de respaldo
 - ota1
 - Permite actualizar sin borrar el anterior
 - Si falla la actualización vuelve al anterior
 - Esto es clave para sistemas industriales / IIoT

d) spiffs:Sistema de archivos SPIFFS

- Guarda archivos
- JSON
- HTML
- CSS
- logs
- modelos pequeños (ej. coeficientes)
- Se mantiene tras reinicios

8. Según la estructura del proyecto(control, IA, IIoT)

- OTA te permite actualizar controladores sin parar planta
- NVS es ideal para:
 - parámetros de control
 - ganancias PI
 - coeficientes AR/IA
- SPIFFS sirve para:
 - logs
 - perfiles
 - datos históricos

9. Versionamiento de firmware ver 2.4

Algoritmo 2.4 Versionamiento

```
;Determinar Versión de Firmware
; X.Y.Z
; (X) versión mayor
; (Y) versión menor
; (Z) revisión
; Alpha, Beta,
;RC (Alpha es una versión inestable
;Beta una versión mas estable que Alpha
; RC (Release Candidate) )
; v1.0.Alpha – v1.0.a0
; v2.1.Beta – v1.0.b1
; v3.0–RC – v3.0.rc3
; Fecha: Año–mes–día ; v1.0.0a–20260202
```

2.2. Esquema del Proyecto

La figura 2.3 muestra de manera general la arquitectura del proyecto para el uso de la esp32 en modo Ap o Cliente.

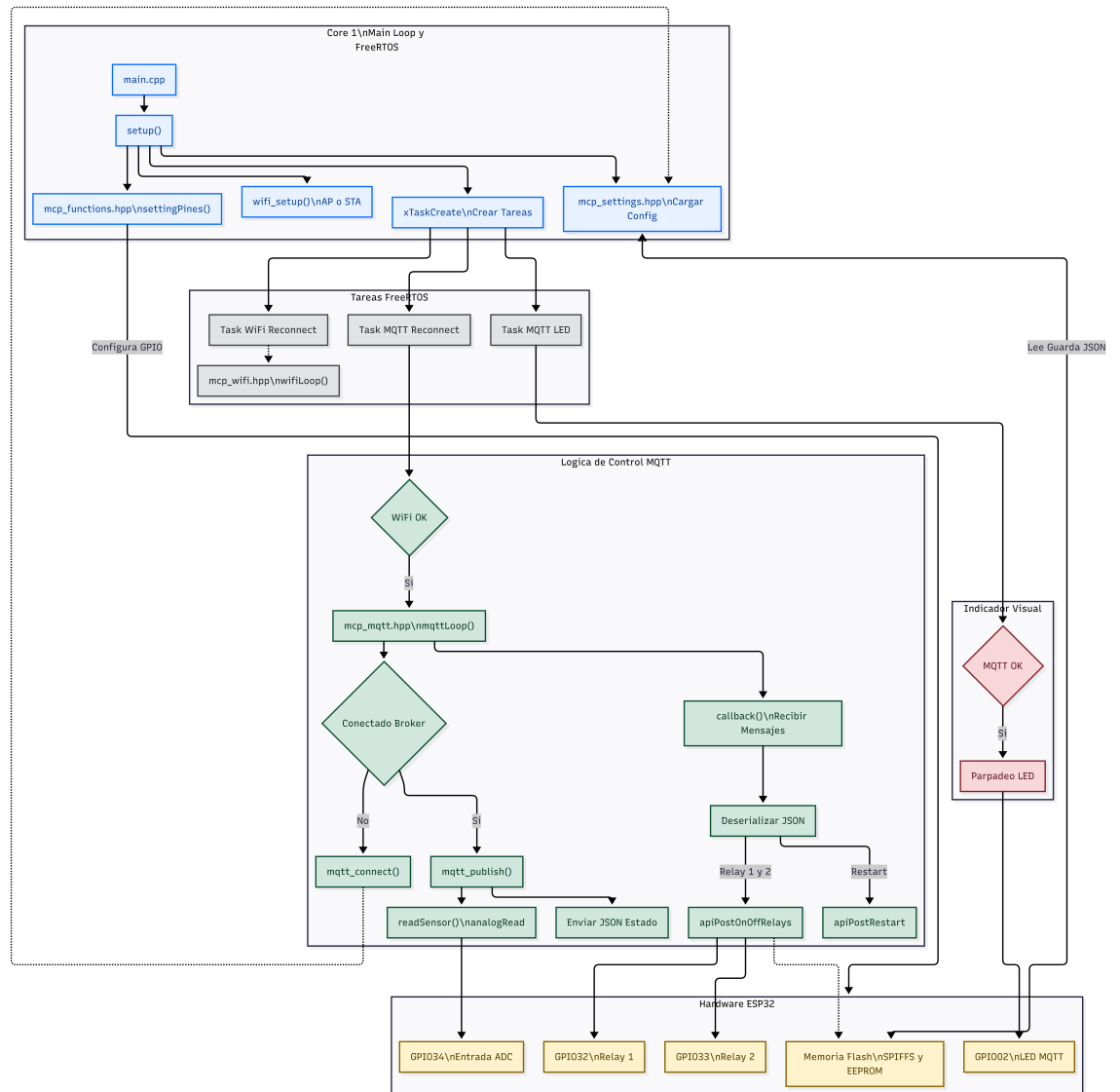


Figura 2.3: Diagrama principal del proyecto

1. La Jerarquía de Red y el Broker MQTT En una arquitectura MQTT, el Broker es el nodo central (Hub). Para que un ESP32 funcione como cliente, debe existir una infraestructura de red subyacente que permita el transporte de capas superiores (TCP/IP).
 - En Modo STA: El ESP32 delega la gestión de la red al Router (Gateway). El Router se encarga de la asignación de IPs vía DHCP, la resolución de nombres (DNS) y, lo más importante, el enrutamiento hacia el exterior. Esto permite que el ESP32 alcance brokers en la nube (como AWS IoT, HiveMQ o Adafruit IO) o en servidores locales (como Raspberry Pi con Mosquitto).
 - En Modo AP: El ESP32 asume el rol de Gateway. Sin embargo, por defecto, un ESP32 en modo AP no tiene una "salida" a internet a menos que actúe como puente (Bridge), lo cual consume recursos críticos de la CPU y memoria RAM, limitando la estabilidad de la conexión MQTT.
2. El Ciclo de Vida de la Conexión (Stack TCP/IP) Técnicamente, el proceso de reconexión que mencionas (TaskMqttReconnect) opera sobre varias capas del modelo OSI:
 - Capa 2 (Enlace): El ESP32 negocia la autenticación WPA2 con el Router.
 - Capa 3 (Red): El stack LwIP (Lightweight IP) del ESP32 obtiene una dirección IP. Sin una IP válida en la subred del Broker, el socket TCP no puede abrirse.
 - Capa 4 (Transporte): Se establece un Handshake TCP en el puerto 1883 (o 8883 para TLS).
 - Capa 7 (Aplicación): Se envía el paquete CONNECT de MQTT.

Si el ESP32 estuviera en modo AP, el Broker tendría que "saltar" a la red generada por el ESP32, lo que rompería la escalabilidad: un Broker suele manejar cientos de clientes; no puede estar saltando de red en red.
3. Gestión de Recursos y Eficiencia El uso de MQTT en modo STA permite aprovechar funciones avanzadas de ahorro de energía:
 - Modem-Sleep: En modo STA, el ESP32 puede entrar en un estado de bajo consumo entre los intervalos de "Keep Alive" de MQTT, manteniendo la asociación con el Router. En modo AP, la radio debe estar activa constantemente para mantener la baliza (Beacon) de la red visible, lo que drena la batería rápidamente.
 - Determinismo en la Tarea: Tu TaskMqttReconnect actúa como una Máquina de Estados. Verifica el bit de evento de WiFi (WIFI_CONNECTED_BIT);

si es verdadero, intenta el `mqtt_client_connect()`. Esta separación de intereses (Separation of Concerns) asegura que el stack de red no bloquee la lógica de tu sensor.

4. El flujo de información bidireccional se muestra en 2.4

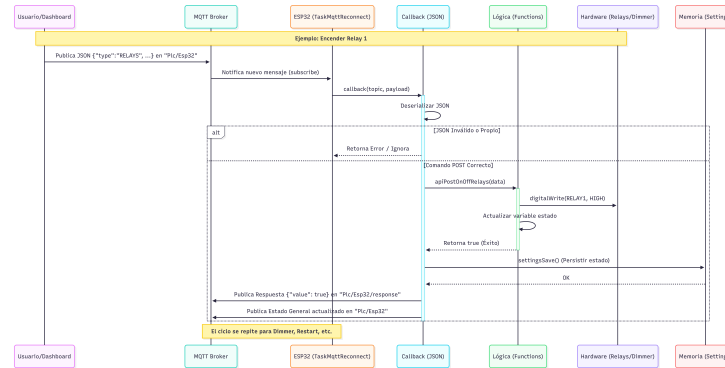


Figura 2.4: Ciclo de Vida de un Mensaje MQTT.

2.3. Programación de protocolos de comunicación

2.3.1. Configuración de variables globales para las conexión wifi

A continuación se describe `mcp_header.hpp`, la cual es una librería de parámetros globales que permite la configuración de las conexiones de protocolo de comunicación ver 2.5

Algoritmo 2.5 Configuraciones globales del dispositivo

```
// -----
// Definiciones
// -----
#define WIFILED 12 // GPIO12 LED WIFI
#define MQTTLED 13 // GPIO13 LED MQTT
// -----
// CALCULAR LA CAPACIDAD DEL JSON
// Asistente ArduinoJson: https://arduinojson.org/v6/assistant/
// Documentación: https://arduinojson.org/v6/api/json/deserializejson/
// -----
const size_t capacitySettings = 1536;
// -----
// Versión de Firmware desde las variables de entorno platformio.ini
// -----
#define TEXTIFY(A) #A
#define ESCAPEQUOTE(A)
TEXTIFY(A)
String device_fw_version = ESCAPEQUOTE(BUILD_TAG);
// -----
// Version de Hardware y Fabricante
// -----
#define device_hw_version "MCP32 v1 00000000" // Versión del hardware
#define device_manufacturer "IOTTSolutions" // Fabricante
// -----
// Zona configuración Dispositivo
// -----
boolean device_config_file; // Identificador para archivo de configuración
char device_config_serial[30]; // Numero de serie de cada Archivo configuración unico
char device_id[30]; // ID del dispositivo
int device_restart; // Número de reinicios
char device_old_user[15]; // Usuario para acceso al servidor Web char
char device_old_password[15]; // Contraseña del usuario servidor Web
// -----
// Zona configuración WIFI modo Cliente
// -----
boolean wifi_ip_static; // Uso de IP Estática DHCP
char wifi_ssid[30]; // Nombre de la red WiFi
char wifi_password[30]; // Contraseña de la Red WiFi
char wifi_ipv4[15]; // Dir IPv4 Estático
char wifi_gateway[15]; // Dir IPv4 Gateway
char wifi_subnet[15]; // Dir IPv4 Subred
char wifi_dns_primary[15]; // Dir IPv4 DNS primario
char wifi_dns_secondary[15]; // Dir IPv4 DNS secundario
// -----
// Zona configuración WIFI modo AP
// -----
boolean ap_mode; // Uso de Modo AP
char ap_ssid[31]; // Nombre del SSID AP
char ap_password[63]; // Contraseña del AP
int ap_channel; // Canal AP
int ap_visibility; // Es visible o no el AP (0 - Visible 1 - Oculto)
int ap_connect; // Número de conexiones en el AP máx 8 conexiones ESP32
// -----
// Zona configuración MQTT
// -----
boolean mqtt_cloud_enable; // Habilitar MQTT Broker
char mqtt_cloud_id[30]; // Cliente ID MQTT Broker
char mqtt_user[30]; // Usuario MQTT Broker
char mqtt_password[39]; // Contraseña del MQTT Broker
char mqtt_server[39]; // Servidor del MQTT Broker
int mqtt_port; // Puerto servidor MQTT Broker
boolean mqtt_retain; // Habilitar mensajes retenidos
int mqtt_qos; // Calidad del servicio
boolean mqtt_time_send; // Habilitar en envio de datos
int mqtt_time_interval; // Tiempo de envio por MQTT en segundos
boolean mqtt_status_send; // Habilitar en envio de estados
// -----
// Zona Firmware Update TODO: Para La API REST
// -----
size_t content_len;
#define U_PART U_SPIFFS // Unidad de almacenamiento para Firmware
// Update U_PART formatea en firmware U_SPIFFS formatea en el SPIFFS
// -----
// Zona EEPROM para contador de reinicios
// -----
#define Start_Address 0
#define Restart_Address Start_Address + sizeof(int)
```

2.3.2. Configuración de Lectura, Escritura y Reset de parámetros.

La librería ArduinoJson permite de manera eficiente en un entorno industrial o de desarrollo con microcontroladores (como un Raspberry Pi PLC o ESP32), configurar de manera inteligente la gestión de la memoria <https://arduinojson.org/v7/assistant/#/step1>.

1. Configuración de Memoria: Static vs Dynamic: La elección del "Document" es el paso más crítico en ArduinoJson 7.

- JsonDocument (Recomendado para V7): En la versión más reciente, ya no necesitas preocuparte por elegir entre Static o Dynamic en la mayoría de los casos; el objeto JsonDocument gestiona el heap de forma automática y eficiente.
- Capacidad: Aunque es dinámico, si conoces el tamaño máximo de tus archivos, es una buena práctica pre-asignar espacio para evitar fragmentación.
- Se debe agregar la primera librería que permita trabajar con archivos Json, se debe configurar el archivo platformio.ini.

- platformio.ini
lib_deps =
bblanchon/ArduinoJson@^6.21.5

Para la configuración de los parámetros de conexión wifi mediante archivos Json tendremos la siguiente arquitectura.

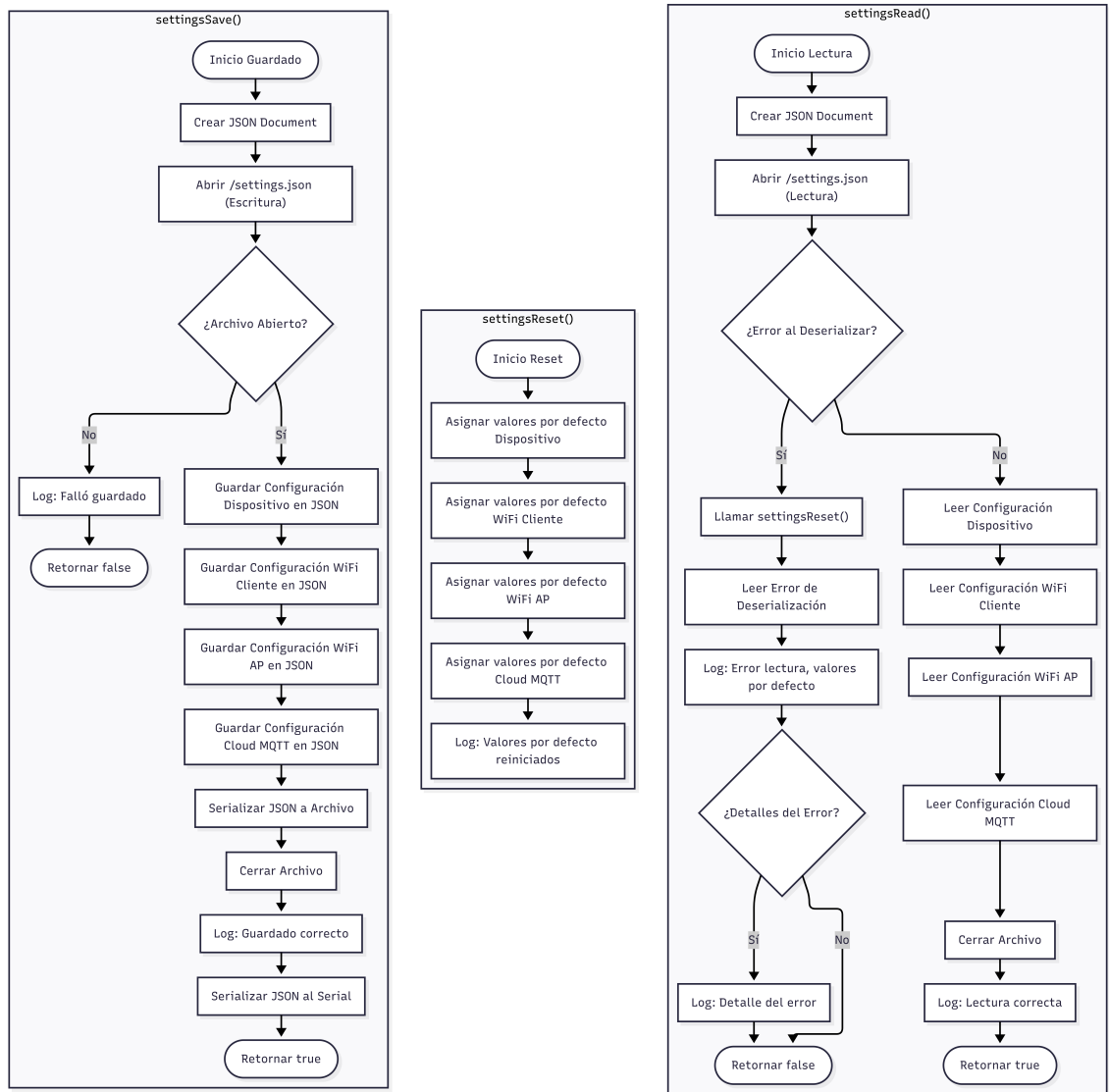


Figura 2.5: Arquitectura de configuración Wiffi

La figura 2.5 muestra las tres funciones asociadas a la configuración del esp32 de la librería mcp_settings.hpp ver 2.6, se muestra las formas de conexión a la red ya sea como usuario o punto de acceso. De la misma manera se establece el protocolo Mqtt Si no llega ninguna configuración toma los parámetros por defecto de fabrica

El código asociado a la función boolean settingsRead se muestra en 2.7.

Algoritmo 2.6 mcp_settings.hpp

```
boolean settingsRead();  
//Leer parámetros  
void settingsReset();  
//Reiniciar parámetros  
boolean settingsSave();  
//Guardar parámetros
```

Algoritmo 2.7 settingsRead

```
boolean settingsRead(){  
    DynamicJsonDocument jsonSettings(capacitySettings);  
    File file = SPIFFS.open("/settings.json", "r");  
  
    if(deserializeJson(jsonSettings, file)){  
        // Tomar los valores de Fábrica  
        settingsReset();  
        DeserializationError error = deserializeJson(jsonSettings, file);  
        log("[ ERROR ] Falló la lectura de las configuraciones, tomando  
            valores por defecto");  
        if(error){  
            log("[ ERROR ] " + String(error.c_str()));  
        }  
        return false;  
    }else{  
        =====  
        // Dispositivo settings.json  
        =====  
        device_config_file = jsonSettings["device_config_file"];  
        strlcpy(device_config_serial, jsonSettings["device_config_serial"], sizeof(  
            device_config_serial));  
        strlcpy(device_id, jsonSettings["device_id"], sizeof(device_id));  
        strlcpy(device_old_user, jsonSettings["device_old_user"], sizeof(device_old_user));  
        strlcpy(device_old_password, jsonSettings["device_old_password"], sizeof(  
            device_old_password));  
        =====  
        // WIFI Cliente settings.json  
        =====  
        wifi_ip_static = jsonSettings["wifi_ip_static"];  
        strlcpy(wifi_ssid, jsonSettings["wifi_ssid"], sizeof(wifi_ssid));  
        strlcpy(wifi_password, jsonSettings["wifi_password"], sizeof(wifi_password));  
        strlcpy(wifi_ipv4, jsonSettings["wifi_ipv4"], sizeof(wifi_ipv4));  
        strlcpy(wifi_subnet, jsonSettings["wifi_subnet"], sizeof(wifi_subnet));  
        strlcpy(wifi_gateway, jsonSettings["wifi_gateway"], sizeof(wifi_gateway));  
        strlcpy(wifi_dns_primary, jsonSettings["wifi_dns_primary"], sizeof(wifi_dns_primary)  
            );  
        strlcpy(wifi_dns_secondary, jsonSettings["wifi_dns_secondary"], sizeof(  
            wifi_dns_secondary));  
        =====  
        // WIFI AP settings.json  
        =====  
        ap_mode = jsonSettings["ap_mode"];  
        strlcpy(ap_ssid, jsonSettings["ap_ssid"], sizeof(ap_ssid));  
        strlcpy(ap_password, jsonSettings["ap_password"], sizeof(ap_password));  
        ap_visibility = jsonSettings["ap_visibility"];  
        ap_chanel = jsonSettings["ap_chanel"];  
        ap_connect = jsonSettings["ap_connect"];  
        =====  
        // Cloud settings.json  
        =====  
        mqtt_cloud_enable = jsonSettings["mqtt_cloud_enable"];  
        strlcpy(mqtt_user, jsonSettings["mqtt_user"], sizeof(mqtt_user));  
        strlcpy(mqtt_password, jsonSettings["mqtt_password"], sizeof(mqtt_password));  
        strlcpy(mqtt_server, jsonSettings["mqtt_server"], sizeof(mqtt_server));  
        strlcpy(mqtt_cloud_id, jsonSettings["mqtt_cloud_id"], sizeof(mqtt_cloud_id));  
        mqtt_port = jsonSettings["mqtt_port"];  
        mqtt_retain = jsonSettings["mqtt_retain"];  
        mqtt_qos = jsonSettings["mqtt_qos"];  
        mqtt_time_send = jsonSettings["mqtt_time_send"];  
        mqtt_time_interval = jsonSettings["mqtt_time_interval"];  
        mqtt_status_send = jsonSettings["mqtt_status_send"];  
        file.close();  
        log("[ INFO ] Lectura de las configuraciones correcta");  
        return true;  
    }  
}
```

El código asociado a la función void settingsReset se muestra en 2.8

Algoritmo 2.8 settingsReset

```
void settingsReset() {  
    // -----  
    // Dispositivo settings.json  
    // -----  
    device_config_file = true;  
    // Habilita el archivo de configuración  
    strcpy(device_config_serial, deviceID().c_str(),  
           sizeof(device_config_serial));  
    strcpy(device_id, "adminmcp", sizeof(device_id));  
    strcpy(device_old_user, "admin", sizeof(device_old_user));  
    strcpy(device_old_password, "admin", sizeof(device_old_password));  
    // -----  
    // WIFI Cliente settings.json  
    // -----  
    wifi_ip_static = jsonSettings["wifi_ip_static"];  
    strcpy(wifi_ssid, jsonSettings["wifi_ssid"],  
           sizeof(wifi_ssid));  
    strcpy(wifi_password, jsonSettings["wifi_password"],  
           sizeof(wifi_password));  
    strcpy(wifi_ipv4, jsonSettings["wifi_ipv4"],  
           sizeof(wifi_ipv4));  
    strcpy(wifi_subnet, jsonSettings["wifi_subnet"],  
           sizeof(wifi_subnet));  
    strcpy(wifi_gateway, jsonSettings["wifi_gateway"],  
           sizeof(wifi_gateway));  
    strcpy(wifi_dns_primary, jsonSettings["wifi_dns_primary"],  
           sizeof(wifi_dns_primary));  
    strcpy(wifi_dns_secondary, jsonSettings["wifi_dns_secondary"],  
           sizeof(wifi_dns_secondary));  
    // -----  
    // WIFI AP settings.json  
    // -----  
    ap_mode = jsonSettings["ap_mode"];  
    strcpy(ap_ssid, jsonSettings["ap_ssid"], sizeof(ap_ssid));  
    strcpy(ap_password, jsonSettings["ap_password"],  
           sizeof(ap_password));  
    ap_visibility = jsonSettings["ap_visibility"];  
    ap_chanel = jsonSettings["ap_chanel"];  
    ap_connect = jsonSettings["ap_connect"];  
    // -----  
    // Cloud settings.json  
    // -----  
    mqtt_cloud_enable = jsonSettings["mqtt_cloud_enable"];  
    strcpy(mqtt_user, jsonSettings["mqtt_user"],  
           sizeof(mqtt_user));  
    strcpy(mqtt_password, jsonSettings["mqtt_password"],  
           sizeof(mqtt_password));  
    strcpy(mqtt_server, jsonSettings["mqtt_server"],  
           sizeof(mqtt_server));  
    strcpy(mqtt_cloud_id, jsonSettings["mqtt_cloud_id"],  
           sizeof(mqtt_cloud_id));  
    mqtt_port = jsonSettings["mqtt_port"];  
    mqtt_retain = jsonSettings["mqtt_retain"];  
    mqtt_qos = jsonSettings["mqtt_qos"];  
    mqtt_time_send = jsonSettings["mqtt_time_send"];  
    mqtt_time_interval = jsonSettings["mqtt_time_interval"];  
    mqtt_status_send = jsonSettings["mqtt_status_send"];  
    file.close();  
    log("[ INFO ] Lectura de las configuraciones correcta");  
    return true;  
}
```

El código asociado a la función void settingsSave se muestra en 2.9

Algoritmo 2.9 settingsSave

```
// Reiniciar parámetros
boolean settingsSave() {
    // StaticJsonDocument<capacitySettings> jsonSettings;
    DynamicJsonDocument jsonSettings(capacitySettings);
    File file =
        SPIFFS.open("/settings.json", "w+");
    if (file) { // Si el archivo se abrió correctamente
        // -----
        // Dispositivo settings.json
        // -----
        jsonSettings["device_config_file"] =
            device_config_file; // Guardar config file
        jsonSettings["device_config_serial"] =
            device_config_serial;
        jsonSettings["device_id"] = device_id;
        jsonSettings["device_old_user"] = device_old_user;
        jsonSettings["device_old_password"] =
            device_old_password; // Guardar old password
        // -----
        // WIFI Cliente settings.json
        // -----
        jsonSettings["wifi_ip_static"] = wifi_ip_static;
        jsonSettings["wifi_ssid"] = wifi_ssid;
        jsonSettings["wifi_password"] = wifi_password;
        jsonSettings["wifi_ipv4"] = wifi_ipv4;
        jsonSettings["wifi_subnet"] = wifi_subnet;
        jsonSettings["wifi_gateway"] = wifi_gateway;
        jsonSettings["wifi_dns_primary"] = wifi_dns_primary;
        jsonSettings["wifi_dns_secondary"] =
            wifi_dns_secondary; // Guardar DNS secundario
        // -----
        // WIFI AP settings.json
        // -----
        jsonSettings["ap_mode"] = ap_mode;
        jsonSettings["ap_ssid"] = ap_ssid;
        jsonSettings["ap_password"] = ap_password;
        jsonSettings["ap_visibility"] = ap_visibility;
        jsonSettings["ap_chanel"] = ap_chanel;
        jsonSettings["ap_connect"] = ap_connect;
        // -----
        // Cloud settings.json
        // -----
        jsonSettings["mqtt_cloud_enable"] =
            mqtt_cloud_enable;
        jsonSettings["mqtt_user"] = mqtt_user;
        jsonSettings["mqtt_password"] = mqtt_password;
        jsonSettings["mqtt_server"] = mqtt_server;
        jsonSettings["mqtt_cloud_id"] = mqtt_cloud_id;
        jsonSettings["mqtt_port"] = mqtt_port;
        jsonSettings["mqtt_retain"] = mqtt_retain;
        jsonSettings["mqtt_qos"] = mqtt_qos;
        jsonSettings["mqtt_time_send"] = mqtt_time_send;
        jsonSettings["mqtt_time_interval"] =
            mqtt_time_interval;
        jsonSettings["mqtt_status_send"] = mqtt_status_send;
        serializeJsonPretty(jsonSettings, file);
        file.close();
        log("[ INFO ] Configuración Guardada correctamente");
        serializeJsonPretty(jsonSettings, Serial);
        return true;
    } else {
        log("[ ERROR ] Falló el guardado de la configuración");
        return false;
    }
}
```

2.3.3. Ciclo de Vida del MQTT

La grafica 2.6 muestra el ciclo de vida del mensaje tanto de la transmisión y recepción de los mensajes mediante el protocolo de comunicación MQTT

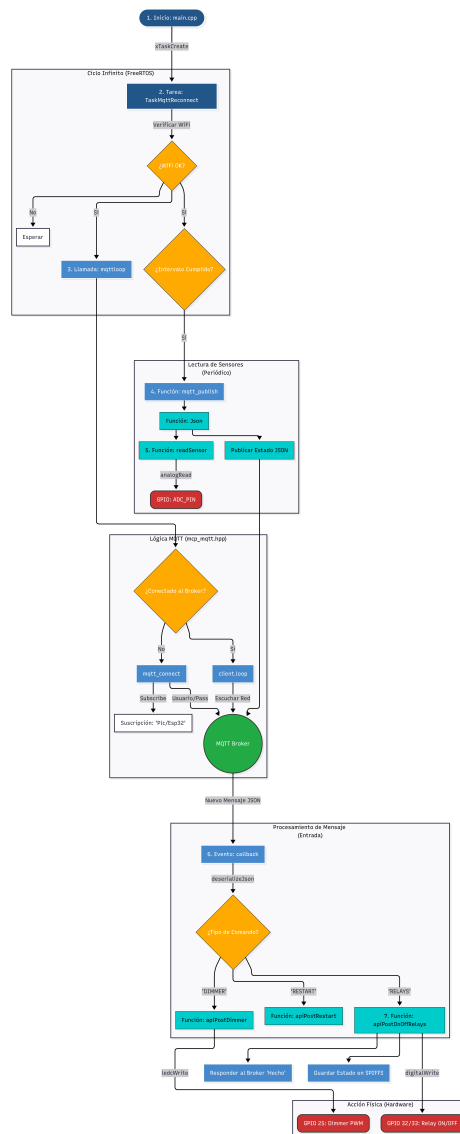


Figura 2.6: Ciclo de vida de Mqtt

Ciclo infinito (FreeRtos)

1. Empieza con una función de FreeRTOS para crear una "Tarea" (un hilo de ejecución), `xTaskCreate(TaskMqttReconnect, "TaskMqttReconnect", 1024 * 6, NULL, 2, NULL)`.

1 TaskMqttReconnect :

- ¿Qué es?: La función que contiene el código de la tarea.
- En resumen: "Oye ESP32, quiero que ejecutes ESTA función en paralelo".

2 "TaskMqttReconnect":

- ¿Qué es?: Un nombre en texto simple para la tarea.
- Para qué sirve: Solo para depuración (debug). Si el sistema falla, te dirá "Falló en la tarea llamada 'TaskMqttReconnect'".

3 1024 * 6 (6144):

- ¿Qué es?: El tamaño de la Memoria Stack (pila) asignada a esta tarea en Bytes.
- Importante: Como tu tarea usa JSON (DynamicJsonDocument), strings y WiFi, necesita bastante memoria. 6KB (6144) es un valor seguro para evitar que se desborde (Stack Overflow) y el ESP32 se reinicie.

4 NULL:

- ¿Qué es?: Parámetros que le podrías pasar a la función.
- En este caso: No necesitas pasarle ninguna variable extra al iniciar, así que se deja vacío (NULL).

5 2:

- ¿Qué es?: La Prioridad de la tarea.
- Escala: Números más altos = Mayor urgencia.
 - La tarea WiFi tiene prioridad 3 (Más importante).
 - Esta tarea MQTT tiene prioridad 2.
 - La tarea del LED tiene prioridad 1 (Menos importante).
 - Significado: Si la CPU está muy ocupada, atenderá primero al WiFi, luego al MQTT y por último al LED.

6 NULL:

- ¿Qué es?: El "Manejador" (Handle) de la tarea.
- Para qué sirve: Si quisieras pausar, reanudar o eliminar esta tarea desde otra parte del código, necesitarías guardar su referencia aquí. Como no planeas manipularla externamente, se deja en NULL.

Algoritmo 2.10 Función TaskMqttReconnect.

```
// mcp_tasks.hpp
void TaskMqttReconnect(void *pvParameters) {
    (void)pvParameters;
    while (1) {
        if ((WiFi.status() == WL_CONNECTED)) {
            if (mqtt_server != 0) {
                mqttloop();
            }
            if (mqttClient.connected()) {
                if (millis() - lasMsg > mqtt_time_interval) {
                    lasMsg = millis();
                    mqtt_publish();
                    log("INFO: Mensaje enviado por MQTT...");
                }
            }
        }
    }
}
```

Esta tarea de FreeRTOS actúa como un monitor persistente que gestiona la comunicación con el exterior. Primero, valida la Capa de Enlace verificando que el WiFi esté conectado; si es así, y existe una dirección de servidor válida, ejecuta `mqttloop()` para procesar los paquetes de red pendientes y mantener latente la sesión con el Broker. Una vez confirmada la conexión activa en la Capa de Aplicación, utiliza un mecanismo de conteo de tiempo no bloqueante (`millis()`) para enviar datos a través de `mqtt_publish()` solo cuando se cumple el intervalo definido, evitando saturar la red y permitiendo que el sistema siga respondiendo a otros procesos sin detenerse.

2. La función 2.10 llama internamente a `mqttloop`.

1 `mqttloop` la función 2.11 hace parte de la librería de configuración `mcp_mqtt.hpp`

Algoritmo 2.11 Función `mqttloop()`.

```
void mqttloop() {
    if (mqtt_cloud_enable) {
        if (!mqttClient.connected()) {
            long now = millis();
            if ((now < 60000) || ((now -
                lastMqttReconnectAttempt) > 120000)) {
                lastMqttReconnectAttempt = now;
                if (mqtt_connect()) {
                    lastMqttReconnectAttempt = 0;
                }
                // setOnSingle(MQTTLED);
            }
        } else {
            mqttClient.loop();
            // setOffSingle(MQTTLED);
        }
    }
}
```

Esta función actúa como el motor de gestión de conectividad del cliente MQTT, operando bajo una lógica de reconexión inteligente y no bloqueante. Su flujo principal valida primero si el servicio está habilitado y verifica el estado de la sesión; si la conexión se ha perdido, emplea un algoritmo de temporización basado en `millis()` que limita los intentos de reconexión (una vez cada 120 minutos tras el primer minuto de arranque), evitando así que el procesador sature el stack de red con peticiones fallidas constantes. Cuando la sesión es exitosa, la función delega el control a `mqttClient.loop()`, que es el método encargado de mantener vivo el "Keep Alive" con el servidor, procesar los mensajes entrantes y gestionar los buffers de salida, asegurando una comunicación fluida y estable con el broker.

2 Se llama entonces a la función `mqtt_connect{()}` la cual se describe en 2.12

Algoritmo 2.12 Función `mqtt_connect()`.

```
boolean mqtt_connect() {
    mqttClient.setServer(mqtt_server, mqtt_port);
    mqttClient.setCallback(callback);
    log("MQTT: Intentando conexión al Broker MQTT...");
    if (mqttClient.connect(mqtt_cloud_id, mqtt_user, mqtt_password,
        mqtt_willTopic.c_str(), mqtt_willQoS, mqtt_willRetain,
        mqtt_willMessage.c_str())) {
        log("INFO: Conectado al Broker MQTT -> " + String(mqtt_server));
    }
    mqttClient.setBufferSize(1024 * 5);
    log("INFO: Buffer MQTT Size: " + String(mqttClient.getBufferSize()
        ()) +
        " Bytes");
    String topic_subscribe = String(mqtt_topic);
    topic_subscribe.toCharArray(topic, 150);
    if (mqttClient.subscribe(topic)) {
        log("INFO: Suscrito al tópico: " + String(topic));
    } else {
        log("ERROR: MQTT - Falló la suscripción");
    }
    String mqtt_willMessageCon =
        "{\"connected\": true, \"username\": \"" + String(
            mqtt_user) + "\"}";
    mqttClient.publish(mqtt_willTopic.c_str(), mqtt_willMessageCon.c_str()
        ());
    } else {
        log("ERROR: MQTT - Falló, código de error = " + String(
            mqttClient.state()));
        return (0);
    }
}
return (1);
}
```

Esta función gestiona el proceso de autenticación y establecimiento de sesión entre el ESP32 y el servidor Mosquitto. Primero, configura el punto de enlace y el puerto, definiendo además una función de callback para procesar los mensajes entrantes. Al intentar la conexión, el código envía no solo las credenciales de usuario, sino también la configuración del Last Will and Testament (LWT), un mecanismo de seguridad que permite al Broker notificar a otros dispositivos si el ESP32 se desconecta inesperadamente. Una vez establecida la sesión, se amplía el buffer de red a 5 KB para permitir el manejo de paquetes de datos de gran tamaño (ideales para telemetría compleja), se suscribe automáticamente al tópico de control Plc/Esp32 y publica un mensaje de estado en formato JSON para confirmar que el sistema está operativo y correctamente identificado. La función `mqttClient.setCallback(callback)` es fundamental ya que es la encargada de procesar el mensaje de entrada sin esta línea, puedes recibir mensajes, pero tu código no sabría qué función debe procesarlos y los ignoraría.

- Registra la función: Le dice a la librería PubSubClient que la función llamada `callback` (que definiste mas abajo) es la "Elegida".
- Automatización: A partir de ese momento, cada vez que el ESP32 reciba un mensaje MQTT de un tópico al que esté suscrito, la librería

interrumpe lo que esté haciendo y ejecuta automáticamente la función callback pasándole el mensaje.

Algoritmo 2.13 Función callback()

```
void callback(char *topic, byte *payload, unsigned int length) {
    String command = "";
    String str_topic(topic);
    for (int16_t i = 0; i < length; i++) {
        command += (char)payload[i];
    }
    command.trim();
    log("INFO: MQTT Tópico --> " + str_topic);
    log("INFO: MQTT Mensaje --> " + command);

    DynamicJsonDocument JsonCommand(1024);
    DeserializationError error = deserializeJson(JsonCommand, command);
    if (error) {
        mqtt_response("Desconocido", "Desconocido", "",
            "{\"msg\": \"\";Error, no es un formato JSON!\"}");
        return;
    }
    // Ignorar mensajes propios (evitar bucle infinito)
    if (JsonCommand.containsKey("deviceMqttId")) {
        return;
    }
    if (!JsonCommand.containsKey("method") || !JsonCommand.containsKey("type")) {
        mqtt_response("Desconocido", "Desconocido", "",
            "{\"msg\": \"\";Error, formato JSON no soportado!\"}");
        return;
    }
    // Manejo de Comandos
    if (strcmp(JsonCommand["method"], "POST") == 0 &&
        strcmp(JsonCommand["type"], "RELAYS") == 0) {
        // {"method": "POST", "type": "RELAYS", "data":{"protocol": "MQTT",
        // "output": "RELAY1", "value": false }}
        if (apiPostOnOffRelays(JsonCommand["data"])) {
            if (settingsSave()) {
                mqtt_response(JsonCommand["method"], JsonCommand["type"],
                    JsonCommand["data"]["output"], "{\"value\": true}");
                mqtt_publish();
            }
        } else {
            if (settingsSave()) {
                mqtt_response(JsonCommand["method"], JsonCommand["type"],
                    JsonCommand["data"]["output"], "{\"value\": false}");
                mqtt_publish();
            }
        }
    } else if (strcmp(JsonCommand["method"], "POST") == 0 &&
        strcmp(JsonCommand["type"], "DIMMER") == 0) {
        // {"method": "POST", "type": "DIMMER", "data":{"protocol": "MQTT",
        // "output": "Dimmer", "value": 50 }}
        apiPostDimmer(JsonCommand["data"]);
        // respuesta al cliente MQTT
        mqtt_response(JsonCommand["method"], JsonCommand["type"],
            JsonCommand["data"]["output"],
            "{\"value\": " + String(dim) + "}");
        mqtt_publish();
    } else if (strcmp(JsonCommand["method"], "POST") == 0 &&
        strcmp(JsonCommand["type"], "RESTART") == 0) {
        // CONFIGURAR EL MQTT MEDIANTE POST
        // {"method": "POST", "type": "RESTART", "origin": "MQTT"}
        // llamar la función con los datos
        mqtt_response(JsonCommand["method"], JsonCommand["type"], "",
            F("{\"restart\": true}"));
        delay(100);
        apiPostRestart(JsonCommand["origin"]);
    } else if (strcmp(JsonCommand["method"], "POST") == 0 &&
        strcmp(JsonCommand["type"], "RESTORE") == 0) {
        // CONFIGURAR EL MQTT MEDIANTE POST
        // {"method": "POST", "type": "RESTORE", "origin": "MQTT"}
        // llamar la función con los datos
        mqtt_response(JsonCommand["method"], JsonCommand["type"], "",
            F("{\"restore\": true}"));
        delay(100);
        apiPostRestore(JsonCommand["origin"]);
    } else {
        mqtt_response("Desconocido", "Desconocido", "",
            "{\"msg\": \"\";Error, no es un comando soportado!\"}");
    }
}
```

Esta función actúa como el centro de control y despacho de comandos del sistema, encargándose de procesar de manera asíncrona todos los mensajes entrantes desde el Broker MQTT. El código transforma el flujo de bytes recibido en un objeto estructurado mediante la librería `ArduinoJson`, implementando primero filtros de seguridad para validar el formato, descartar mensajes mal formados y prevenir bucles infinitos al ignorar ecos del propio dispositivo. Una vez validada la estructura, el callback opera como un intérprete de protocolos API sobre MQTT, discriminando entre diferentes métodos (como POST) y tipos de carga (RELAYS, DIMMER, RESTART o RESTORE); esto permite ejecutar acciones de control físico, actualizar el estado de actuadores, guardar configuraciones en la memoria no volátil y emitir respuestas de confirmación en tiempo real hacia la base de datos o el cliente, garantizando una arquitectura de control bidireccional robusta y escalable.

3 dentro de 2.13 hay dos funciones `mqtt_response` 2.14y `mqtt_publish` 2.15.

Algoritmo 2.14 Función `mqtt_response()`

```
void mqtt_response(String method, String type, String msg, String value) {
    String data = "";
    DynamicJsonDocument jsonDoc(10240);
    DynamicJsonDocument jsonData(10240);
    deserializeJson(jsonData, value);
    jsonDoc["method"] = method;
    jsonDoc["type"] = type;
    JsonObject dataObj = jsonDoc.createNestedObject("data");
    dataObj["msg"] = msg;
    dataObj["value"] = jsonData;
    serializeJson(jsonDoc, data);
    String topic = "Plc/Esp32"; // Simulando PathMqttTopic("response")
    mqttClient.publish(topic.c_str(), data.c_str()); }

```

Esta función `mqtt_response` se encarga de construir y enviar una respuesta estructurada en formato JSON a través de MQTT, normalmente como reacción a un comando o solicitud recibida por el ESP32. Primero crea dos documentos JSON dinámicos: uno para el mensaje final (`jsonDoc`) y otro para interpretar el contenido recibido en el parámetro `value`. Luego deserializa `value` para convertirlo en un objeto JSON válido, lo inserta dentro del campo `data.value` y añade metadatos como el método (`method`), el tipo de mensaje (`type`) y un mensaje descriptivo (`msg`). Finalmente, el objeto completo se serializa a una cadena y se publica en el tópico MQTT `Plc/Esp32`, permitiendo que otros sistemas (backend, dashboards o servicios de control) reciban una respuesta clara, estructurada y consistente

sobre el estado o resultado de la operación ejecutada por el dispositivo.

Algoritmo 2.15 Función `mqtt_publish()`

```
void mqtt_publish() {  
    String topic = String(mqtt_topic);  
    mqtt_data = Json();  
    mqttClient.publish(topic.c_str(), mqtt_data.c_str());  
    mqtt_data = "";  
}
```

Esta función `mqtt_publish` se encarga de publicar periódicamente el estado o los datos actuales del ESP32 en el broker MQTT. Primero construye el tópico de publicación a partir de la variable `mqtt_topic`, asegurando que el mensaje se envíe al canal correcto. Luego genera el contenido del mensaje llamando a la función `Json()`, que encapsula la información del dispositivo (como lecturas de sensores, estados de relés o indicadores del sistema) en formato JSON. Ese mensaje se publica usando el cliente MQTT y, una vez enviado, la variable `mqtt_data` se limpia para liberar memoria y evitar reutilizar datos antiguos. De esta forma, la función mantiene una comunicación ordenada y consistente entre el ESP32 y los sistemas suscriptores, como el backend o el panel de monitoreo.

- 4 La función `Json` es creada con el propósito de enviar la información asociada al dispositivos 2.16.

Algoritmo 2.16 Json()

```
String Json() {
    String response;
    DynamicJsonDocument jsonDoc(3000);
    readSensor();
    jsonDoc["deviceMqttId"] = mqtt_cloud_id;
    jsonDoc["deviceSerial"] = deviceID();
    jsonDoc["deviceManufacturer"] = device_manufacturer;
    jsonDoc["deviceFwVersion"] = device_fw_version;
    jsonDoc["deviceHwVersion"] = device_hw_version;
    jsonDoc["deviceSdk"] = ESP.getSdkVersion();
    JsonObject dataObj = jsonDoc.createNestedObject("data");
    dataObj["deviceRamSizeKB"] = ESP.getHeapSize() / 1024;
    dataObj["deviceRamAvailableKB"] = ESP.getFreeHeap() / 1024;
    dataObj["deviceSpiffsSizeKB"] = SPIFFS.totalBytes() / 1024;
    dataObj["deviceSpiffsUsedKB"] = SPIFFS.usedBytes() / 1024;
    dataObj["deviceActiveTimeSeconds"] = longTimeStr(millis() / 1000);
    dataObj["deviceCpuClockMhz"] = getCpuFrequencyMhz();
    dataObj["deviceFlashSizeMB"] = ESP.getFlashChipSize() / (1024.0 * 1024);
    // Valores Hardware Reales
    dataObj["deviceRelay1Status"] = RELAY1_STATUS ? true : false;
    dataObj["deviceRelay2Status"] = RELAY2_STATUS ? true : false;
    dataObj["deviceDimmer"] = dim;    dataObj["deviceCpuTempC"] = TempCPUValue();
    dataObj["deviceADCValue"] = adcValue;
    dataObj["deviceRestarts"] = device_restart;
    dataObj["wifiRssiStatus"] = WiFi.RSSI();
    dataObj["wifiQuality"] = getRSSIasQuality(WiFi.RSSI());
    dataObj["wifiIPv4"] = ipStr(WiFi.localIP());
    jsonDoc["jsonVersion"] = "1.0.0";
    serializeJson(jsonDoc, response);    return response; }
```

Esta función Json se encarga de recopilar el estado completo del ESP32 y serializarlo en un mensaje JSON listo para ser enviado, normalmente vía MQTT. Al inicio ejecuta readSensor() para actualizar los valores físicos más recientes, como el ADC. Luego construye un objeto JSON que incluye información de identificación del dispositivo (ID MQTT, número de serie, fabricante y versiones de firmware, hardware y SDK), junto con métricas del sistema como uso de memoria RAM, almacenamiento SPIFFS, tiempo activo, frecuencia de CPU y tamaño de la flash. Dentro del bloque data también se agregan los valores reales de hardware, como el estado de los relés, el dimmer, la temperatura del CPU, la lectura ADC y estadísticas de red WiFi (RSSI, calidad de señal e IP). Finalmente, se añade una versión del esquema JSON, se serializa todo a una cadena y se retorna, proporcionando una instantánea completa y estructurada del estado del

dispositivo para monitoreo, diagnóstico o análisis remoto.